



EECS 453

Principles of

Machine Learning

Lecture 4 – Linear Classifier, LDA & Logistic Models

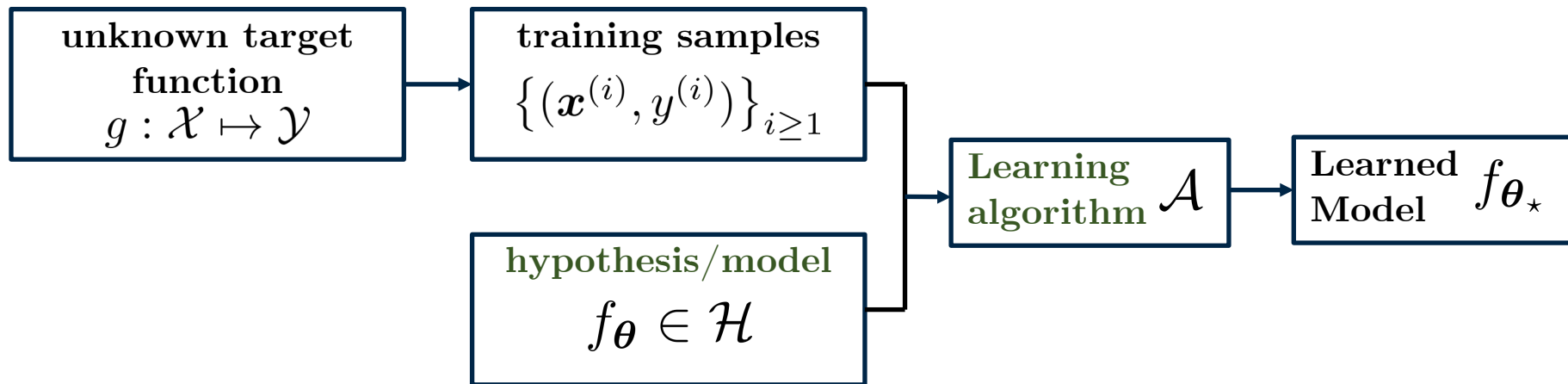
Instructor: Prof. Qing Qu

Lecture Agenda

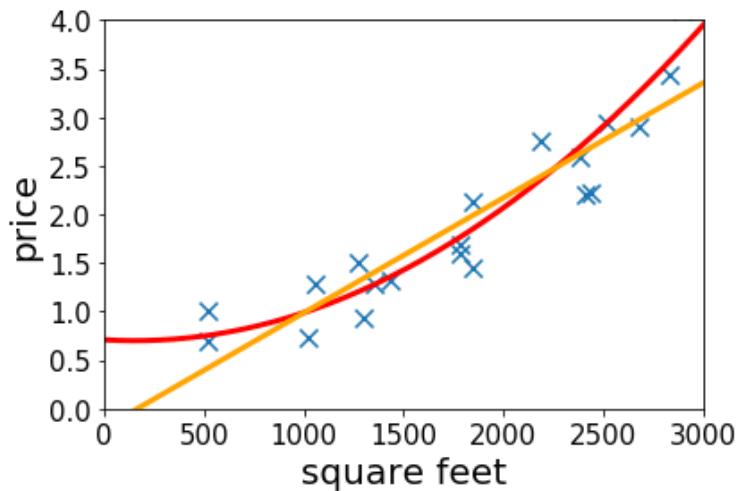
- **Classification Problem & Bayes-Optimal Classifier**
- Linear Discriminative Analysis & Linear Classifier
- Logistic Regression Models

Supervised Learning

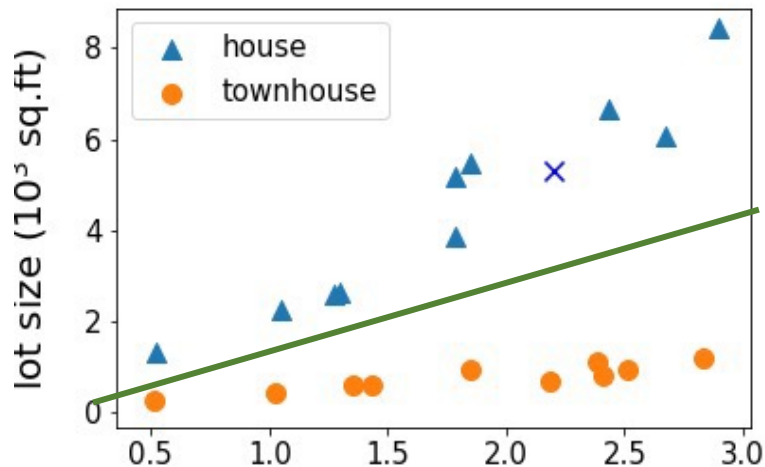
Seen pictorially, the process is therefore like the following



Classification vs. Regression



Regression: the prediction y is *continuous*



Classification: the prediction y is *discrete*

Binary Classification Problem

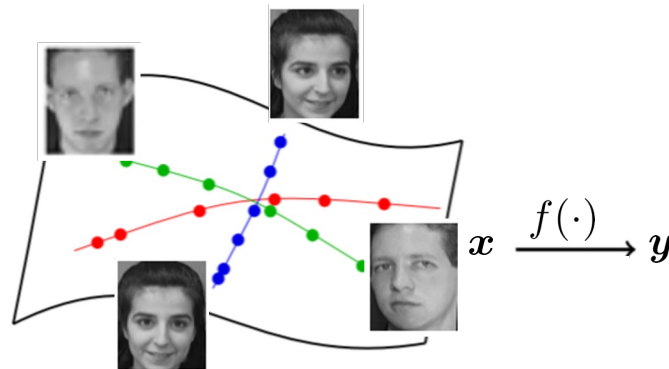
- **Two classes:** $\mathcal{Y} = \{0, 1\}$
- Our goal is to learn to classify two types of examples:
 - Class 0 – labeled as 0,
 - Class 1 – labeled as 1
- We would like to learn $f : \mathcal{X} \mapsto \{0, 1\}$
- Misclassification **error function**, with $y_i = g(\mathbf{x}_i)$

$$e(f(\mathbf{x}_i), y_i) = \mathbb{1}_{f(\mathbf{x}_i) \neq y_i} = \begin{cases} 1 & f(\mathbf{x}_i) \neq y_i \\ 0 & f(\mathbf{x}_i) = y_i \end{cases}$$

Multi-Class Classification Problem

- **Multi-Class Labels:** $k = 0, 1, \dots, K - 1$
 - $K = 10$ classes (MNIST, CIFAR10, etc.)
 - $K = 1000$ classes (ImageNet)
- We would like to learn $f : \mathcal{X} \mapsto [K - 1]$
- Misclassification error function

$$e(f(\mathbf{x}_i), y_i) = \mathbb{1}_{f(\mathbf{x}_i) \neq y_i} = \begin{cases} 1 & f(\mathbf{x}_i) \neq y_i \\ 0 & f(\mathbf{x}_i) = y_i \end{cases}$$



Empirical Risk Minimization (ERM)

- Recall the **definition of risks**

➤ Empirical risk $\hat{R}_n(f_j) := \frac{1}{n} \sum_{i=1}^n e(f_j(\mathbf{x}_i), y_i) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{f_j(\mathbf{x}_i) \neq y_i}$

➤ Population risk $R(f_j) := \mathbb{E}_{\mathbf{x}} [e(f_j(\mathbf{x}), y)] = \mathbb{P}(f_j(\mathbf{x}) \neq y)$

- **Ideally**, we would like to choose **our classifier**:

$$f_{\star} = \arg \min_{f_j \in \mathcal{H}} R(f_j)$$

- If $\hat{R}_n(f_j)$ is a **good estimate** of $R(f_j)$

$$f_{\#} = \arg \min_{f_j \in \mathcal{H}} \hat{R}_n(f_j)$$

The Bayes-Optimal Classifier

Theorem (Bayes-Optimal Classifier): the classifier

$$f_{\star}(\mathbf{x}) := \arg \max_{y \in \mathcal{Y}} \mathbb{P}(y \mid \mathbf{x}) = \arg \max_k p_{y|x}(k|\mathbf{x})$$

is optimal over *all possible* classifiers, in the sense that

$$R_{\star} = R(f_{\star}) \leq R(f), \quad \forall f \in \mathcal{H}$$

- Here, $f : \mathbb{R}^d \rightarrow [K]$ is a classifier with probability of error/risk given by

$$R(f) := \mathbb{P}[f(\mathbf{x}) \neq \mathbf{y}]$$

- Terminology:
 - $f_{\star}$ is called a **Bayes classifier**, $R_{\star}$ is called the **Bayes risk**
 - $p_{y|x}(k|\mathbf{x})$ is called a **posteriori probability** of y

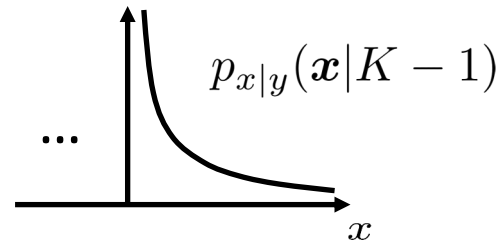
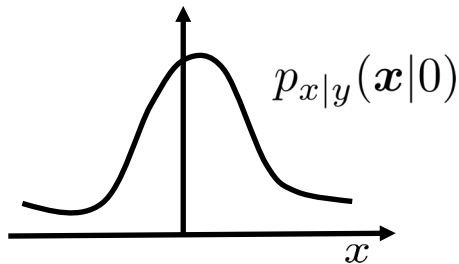
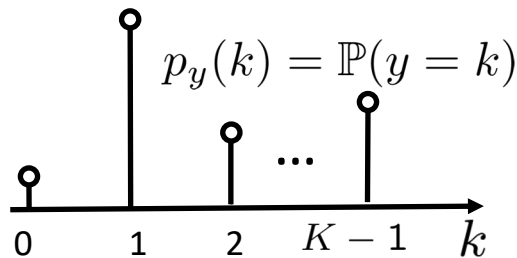
Conditional Probability for Classification

- Classify “cat” and “dog”
- Labeling: ‘Cat’ $y = 1$, ‘Dog’ $y = 0$
- Now you have a test data $\mathbf{x}$ without labeling
- Anyway, suppose you know:
$$\mathbb{P}(y = 1 \mid \mathbf{x}) = 0.8, \quad \mathbb{P}(y = 0 \mid \mathbf{x}) = 0.2.$$
- Which class will you assign $\mathbf{x}$ to?



Conditional Probability and Bayes Rule

- Consider $(\mathbf{x}, y)$, where we suppose
 - $y \in [K - 1] := \{0, 1, \dots, K - 1\}$ is a random variable
 - $\mathbf{x}$ is a random vector in $\mathbb{R}^d$ (depending on y)
- We denote:
 - $p_{x,y}(\mathbf{x}, k)$: the **joint distribution** of $(\mathbf{x}, y)$
 - $p_y(k) = \mathbb{P}(y = k)$: the ***a priori* probability** of Class k
 - $p_{x|y}(\mathbf{x}|k)$: the **class conditional pdf** (i.e., the pdf of $\mathbf{x} \mid y = k$)



Conditional Probability and Bayes Rule

- Consider $(\mathbf{x}, y)$, where we suppose
 - $y \in [K - 1] := \{0, 1, \dots, K - 1\}$ is a random variable
 - $\mathbf{x}$ is a random vector in $\mathbb{R}^d$ (depending on y)
- We denote:
 - $p_{x,y}(\mathbf{x}, k)$: the **joint distribution** of $(\mathbf{x}, y)$
 - $p_y(k) = \mathbb{P}(y = k)$: the ***a priori* probability** of Class k
 - $p_{x|y}(\mathbf{x}|k)$: the **class conditional pdf** (i.e., the pdf of $\mathbf{x} \mid y = k$)

$$p_{x,y}(\mathbf{x}, k) = p_y(k) \cdot p_{x|y}(\mathbf{x} \mid k)$$

The Bayes Rule

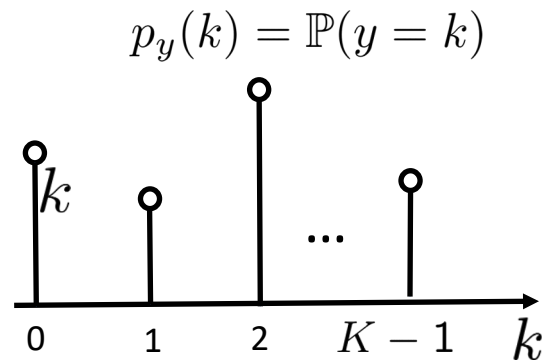
Alternatively, we can factor

$$p_{x,y}(\mathbf{x}, k) = p_x(\mathbf{x}) \cdot p_{y|x}(k|\mathbf{x})$$

- $p_{y|x}(k|\mathbf{x}) = \mathbb{P}(y = k \mid x = \mathbf{x})$ is the *posteriori probability* of class k
- $p_x(\mathbf{x})$ is simply the **marginal pdf** of x

Bayes Rule:

$$\begin{aligned} p_{y|x}(k|\mathbf{x}) &:= \mathbb{P}(y = k | x = \mathbf{x}) \\ &= \frac{p_y(k) \cdot p_{x|y}(\mathbf{x}|k)}{p_x(\mathbf{x})} \end{aligned}$$



The Bayes-Optimal Classifier

Theorem (Bayes-Optimal Classifier): the classifier

$$f_{\star}(\mathbf{x}) := \arg \max_{y \in \mathcal{Y}} \mathbb{P}(y \mid \mathbf{x}) = \arg \max_k p_{y|x}(k|\mathbf{x})$$

is optimal over *all possible* classifiers, in the sense that

$$R_{\star} = R(f_{\star}) \leq R(f), \quad \forall f \in \mathcal{H}$$

- Here, $f : \mathbb{R}^d \rightarrow [K]$ is a classifier with probability of error/risk given by

$$R(f) := \mathbb{P}[f(\mathbf{x}) \neq \mathbf{y}]$$

- Terminology:
 - $f_{\star}$ is called a **Bayes classifier**, $R_{\star}$ is called the **Bayes risk**
 - $p_{y|x}(k|\mathbf{x})$ is called a **posteriori probability** of y

The Bayes-Optimal Classifier

Theorem (Bayes-Optimal Classifier): the classifier

$$f_{\star}(\mathbf{x}) := \arg \max_{y \in \mathcal{Y}} \mathbb{P}(y \mid \mathbf{x}) = \arg \max_k p_{y|\mathbf{x}}(k|\mathbf{x})$$

is optimal over *all possible* classifiers, in the sense that

$$R_{\star} = R(f_{\star}) \leq R(f), \quad \forall f \in \mathcal{H}$$

Issue?

- How can you know $p_{y|\mathbf{x}}(k|\mathbf{x})$?
- Suppose we have training data $\mathcal{S} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, can we find a **natural estimator** for $p_{y|\mathbf{x}}(k|\mathbf{x})$?

Lecture Agenda

- Classification Problem & Bayes-Optimal Classifier
- **Linear Discriminative Analysis & Linear Classifier**
- Logistic Regression Models

Linear Discriminant Analysis (LDA)

To estimate $p_{y|x}(k|\mathbf{x})$, let us make a (strong) **assumption** that

$$\mathbf{x} \mid y = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}), \quad \text{for } k = 0, 1, \dots, K - 1$$

Here, $\mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma})$ denotes multi-variate Gaussian distribution with mean $\boldsymbol{\mu}_k$ and covariance matrix $\boldsymbol{\Sigma}$

$$\mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \right)$$

Note: each class has the same covariance matrix $\boldsymbol{\Sigma}$

Linear Discriminant Analysis (LDA)

To estimate $p_{y|x}(k|\mathbf{x})$, let us make a (strong) **assumption** that

$$\mathbf{x} \mid y = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}), \quad \text{for } k = 0, 1, \dots, K - 1$$

Note: each class has the same covariance matrix $\boldsymbol{\Sigma}$

For LDA, we assume:

- The prior probabilities $p_y(k) = \mathbb{P}(y = k)$, the mean vectors $\boldsymbol{\mu}_k$, and the covariance matrix $\boldsymbol{\Sigma}$ are *all unknown*;
- We estimate them from the *training data*.

The Bayes-Optimal Classifier

Theorem (Bayes-optimal Classifier): the classifier

$$f_{\star}(\mathbf{x}) := \arg \max_{y \in \mathcal{Y}} \mathbb{P}(y \mid \mathbf{x}) = \arg \max_k p_{y|x}(k|\mathbf{x})$$

is optimal over *all possible* classifiers, in the sense that

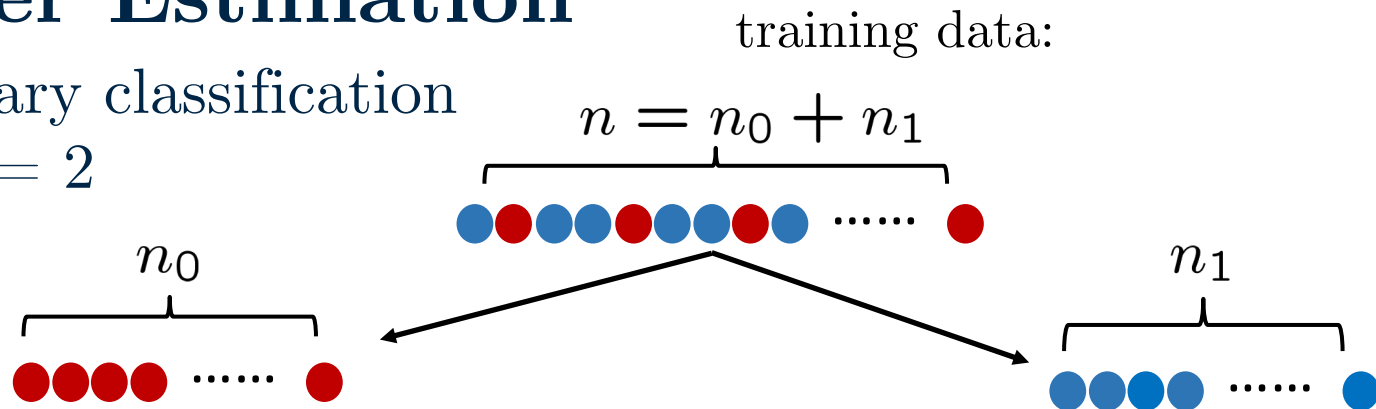
$$R_{\star} = R(f_{\star}) \leq R(f), \quad \forall f \in \mathcal{H}$$

Idea: given training samples $\mathcal{S} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$,
estimate

$$\begin{aligned} p_{y|x}(k|\mathbf{x}) &:= \mathbb{P}(y = k | x = \mathbf{x}) \\ &= \frac{p_y(k) \cdot p_{x|y}(\mathbf{x}|k)}{p_x(\mathbf{x})} \end{aligned}$$

Parameter Estimation

Consider the binary classification problem with $K = 2$



- Estimation of prior probability $p_y(k)$: $\hat{p}_y(0) = \frac{n_0}{n}$, $\hat{p}_y(1) = \frac{n_1}{n}$
- Estimation of sample mean μ_k : $\hat{\mu}_0 = \frac{1}{n_0} \sum_{i:y_i=0} \mathbf{x}_i$, $\hat{\mu}_1 = \frac{1}{n_1} \sum_{i:y_i=1} \mathbf{x}_i$,
- Estimation of sample covariance matrix:

$$\hat{\Sigma}_0 = \frac{1}{n_0} \sum_{i:y_i=0} (\mathbf{x}_i - \hat{\mu}_0) (\mathbf{x}_i - \hat{\mu}_0)^\top \quad \hat{\Sigma}_1 = \frac{1}{n_1} \sum_{i:y_i=1} (\mathbf{x}_i - \hat{\mu}_1) (\mathbf{x}_i - \hat{\mu}_1)^\top$$

Parameter Estimation

Consider the binary classification problem with $K = 2$

- Estimation of prior probability $p_y(k)$: $\hat{p}_y(0) = \frac{n_0}{n}$, $\hat{p}_y(1) = \frac{n_1}{n}$
- Estimation of sample mean μ_k : $\hat{\mu}_0 = \frac{1}{n_0} \sum_{i:y_i=0} \mathbf{x}_i$, $\hat{\mu}_1 = \frac{1}{n_1} \sum_{i:y_i=1} \mathbf{x}_i$,
- Estimation of sample covariance matrix:

$$\hat{\Sigma}_0 = \frac{1}{n_0} \sum_{i:y_i=0} (\mathbf{x}_i - \hat{\mu}_0)(\mathbf{x}_i - \hat{\mu}_0)^\top \quad \hat{\Sigma}_1 = \frac{1}{n_1} \sum_{i:y_i=1} (\mathbf{x}_i - \hat{\mu}_1)(\mathbf{x}_i - \hat{\mu}_1)^\top$$



$$\hat{\Sigma} = \hat{p}_y(0)\hat{\Sigma}_0 + \hat{p}_y(1)\hat{\Sigma}_1$$

“pooled covariance estimate”

Resulting Classifier

For binary classification, the LDA classifier is then:

$$\begin{aligned}\hat{f}(\mathbf{x}) &= \arg \max_{k \in \{0,1\}} \hat{p}_y(k) \cdot \hat{p}_{x|y}(\mathbf{x} \mid k) \\ &= \arg \max_{k \in \{0,1\}} \log(\hat{p}_y(k)) + \log(\hat{p}_{x|y}(\mathbf{x} \mid k))\end{aligned}$$

Given by

$$\begin{aligned}p_{y|x}(k|\mathbf{x}) &:= \mathbb{P}(y = k | x = \mathbf{x}) \\ &= \frac{p_y(k) \cdot p_{x|y}(\mathbf{x}|k)}{p_x(\mathbf{x})}\end{aligned}$$

Resulting Classifier

For binary classification, the LDA classifier is then:

$$\begin{aligned}\hat{f}(\mathbf{x}) &= \arg \max_{k \in \{0,1\}} \log(\hat{p}_y(k)) + \log(\hat{p}_{x|y}(\mathbf{x} | k)) \\ &= \arg \max_{k \in \{0,1\}} \log(\hat{p}_y(k)) + \log \psi(\mathbf{x}; \hat{\mathbf{u}}_k, \hat{\Sigma}) \\ &= \arg \max_{k \in \{0,1\}} \log(\hat{p}_y(k)) - \frac{1}{2} (\mathbf{x} - \hat{\boldsymbol{\mu}}_k)^\top \hat{\Sigma}^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}}_k) \\ &= \arg \min_{k \in \{0,1\}} \underbrace{(\mathbf{x} - \hat{\boldsymbol{\mu}}_k)^\top \hat{\Sigma}^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}}_k)}_{\text{squared } \textit{Mahalanobis distance} \\ \text{between } \mathbf{x} \text{ and } \boldsymbol{\mu}} - 2 \log(\hat{p}_y(k))\end{aligned}$$

$$d_M(\mathbf{x}; \boldsymbol{\mu}, \Sigma) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Resulting Classifier

For binary classification, given the input $\mathbf{x}$:

- We assign $\mathbf{x}$ to Class 0, if

$$(\mathbf{x} - \hat{\boldsymbol{\mu}}_0)^\top \hat{\boldsymbol{\Sigma}}^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}}_0) - 2 \log(\hat{p}_y(0)) < (\mathbf{x} - \hat{\boldsymbol{\mu}}_1)^\top \hat{\boldsymbol{\Sigma}}^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}}_1) - 2 \log(\hat{p}_y(1))$$

- We assign $\mathbf{x}$ to Class 1, if

$$(\mathbf{x} - \hat{\boldsymbol{\mu}}_0)^\top \hat{\boldsymbol{\Sigma}}^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}}_0) - 2 \log(\hat{p}_y(0)) > (\mathbf{x} - \hat{\boldsymbol{\mu}}_1)^\top \hat{\boldsymbol{\Sigma}}^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}}_1) - 2 \log(\hat{p}_y(1))$$

- In sum, we assign $\mathbf{x}$ to the class with
 - Small Mahalanobis distance
 - Large prior probability

Simplification of the Comparison

- It turns out that by setting:

$$\mathbf{w} = \hat{\Sigma}^{-1} (\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0) \in \mathbb{R}^d$$

$$b = \frac{1}{2} \hat{\boldsymbol{\mu}}_0^\top \hat{\Sigma}^{-1} \hat{\boldsymbol{\mu}}_0 - \frac{1}{2} \hat{\boldsymbol{\mu}}_1^\top \hat{\Sigma}^{-1} \hat{\boldsymbol{\mu}}_1 + \log \frac{\hat{p}_y(1)}{\hat{p}_y(0)}$$

- Then we can rewrite our classifier is

$$\mathbf{w}^\top \mathbf{x} + b \underset{1}{\overset{0}{\gtrless}} 0 \quad \text{Linear classifier}$$

When is LDA Appropriate?



Quadratic Discriminant Analysis (QDA)

What happens if the generative model is

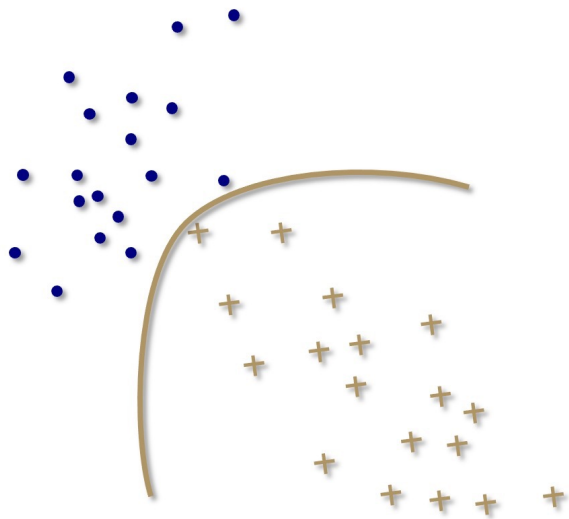
$$\mathbf{x} \mid y = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \text{for } k = 0, 1, \dots, K - 1 \text{ ?}$$

Note: each class has **different** covariance matrices $\boldsymbol{\Sigma}_k$

Set
$$\hat{\boldsymbol{\Sigma}}_k = \frac{1}{|\{i : y_i = k\}|} \sum_{i: y_i = k} (\mathbf{x}_i - \hat{\mathbf{u}}_k) (\mathbf{x}_i - \hat{\mathbf{u}}_k)^\top$$

Proceed as before, only this case the decision boundaries will be *quadratic*

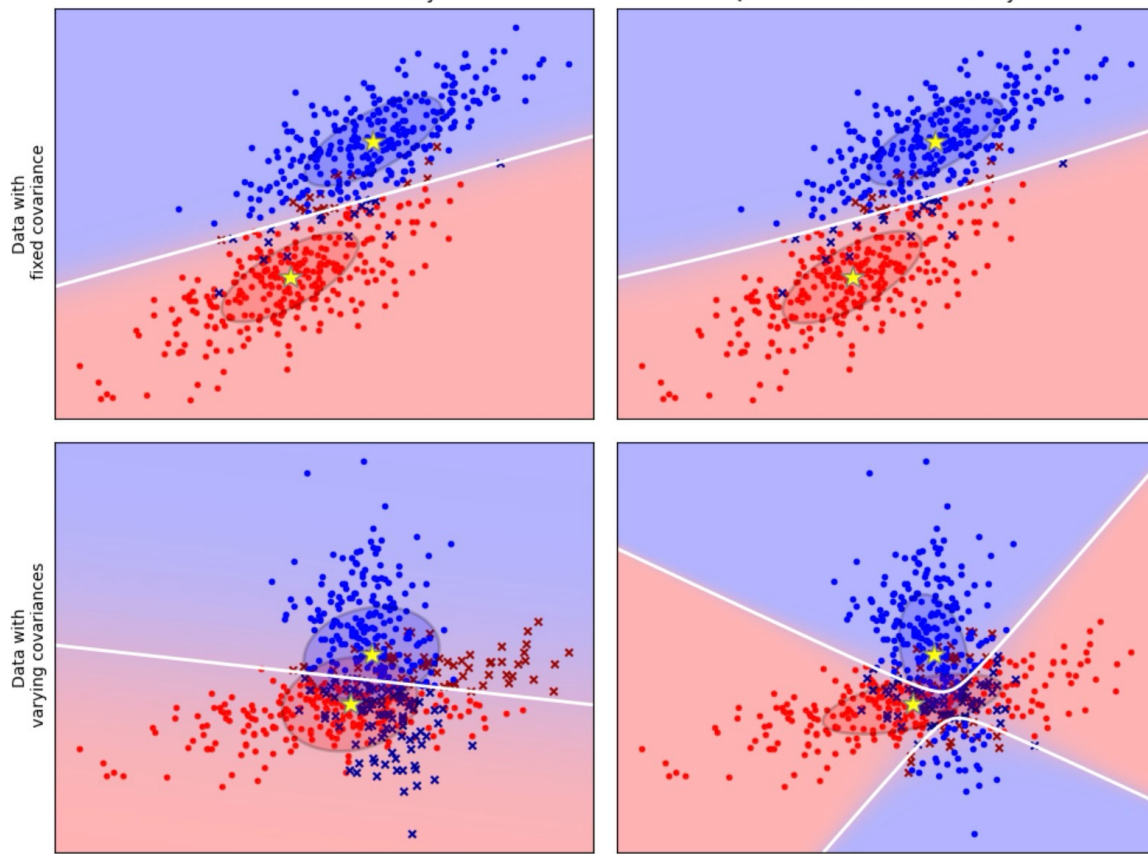
LDA vs. QDA



Linear Discriminant Analysis vs Quadratic Discriminant Analysis

Linear Discriminant Analysis

Quadratic Discriminant Analysis



Summary of LDA

- **Generative models/methods:** the method is based the Gaussian assumption of the data, which is **rarely valid** in practice
- **Estimating large number of parameters:** (class number K , feature dimension d , training samples n)
 - Class prior probabilities: K
 - Class means: Kd
 - Covariance matrices: $d(d+1)/2$
- **Require large number of training samples n :**
 - If d is small and n is large, the estimation could be accurate;
 - If n is small and d is large, “overfitting”, more parameters than observations, poor estimates

Summary of LDA

Theorem (Bayes-optimal Classifier): the classifier

$$f_{\star}(\mathbf{x}) := \arg \max_{y \in \mathcal{Y}} \mathbb{P}(y | \mathbf{x}) = \arg \max_k p_{y|x}(k|\mathbf{x})$$

is optimal over *all possible* classifiers, in the sense that

$$R_{\star} = R(f_{\star}) \leq R(f), \quad \forall f \in \mathcal{H}$$

$$\begin{aligned} p_{y|x}(k|\mathbf{x}) &:= \mathbb{P}(y = k | x = \mathbf{x}) \\ &= \frac{p_y(k) \cdot p_{x|y}(\mathbf{x}|k)}{p_x(\mathbf{x})} \end{aligned}$$

Summary of LDA

$$p_{y|x}(k|\mathbf{x}) := \mathbb{P}(y = k|x = \mathbf{x}) = \frac{p_y(k) \cdot p_{x|y}(\mathbf{x}|k)}{p_x(\mathbf{x})}$$

- LDA estimates by estimating $p_y(k) \cdot p_{x|y}(\mathbf{x}|k)$, which is essentially the *full joint distribution* of $(\mathbf{x}, y)$
- All we *really* need is to be able to estimate $p_{y|x}(k|\mathbf{x})$
 - We do NOT need to know $p_x(\mathbf{x})$, $p_y(k)$, and $p_{x|y}(\mathbf{x}|k)$
- LDA commits one of the cardinal sins of machine learning:
*Never solve a more difficult problem
as an intermediate step*

Lecture Agenda

- Classification Problem & Bayes-Optimal Classifier
- Linear Discriminative Analysis & Linear Classifier
- **Logistic Regression Models**
 - Logistic Function & Linear Classifier
 - Parameter Estimation for Logistic Regression

Lecture Agenda

- Classification Problem & Bayes-Optimal Classifier
- Linear Discriminative Analysis & Linear Classifier
- **Logistic Regression Models**
 - **Logistic Function & Linear Classifier**
 - Parameter Estimation for Logistic Regression

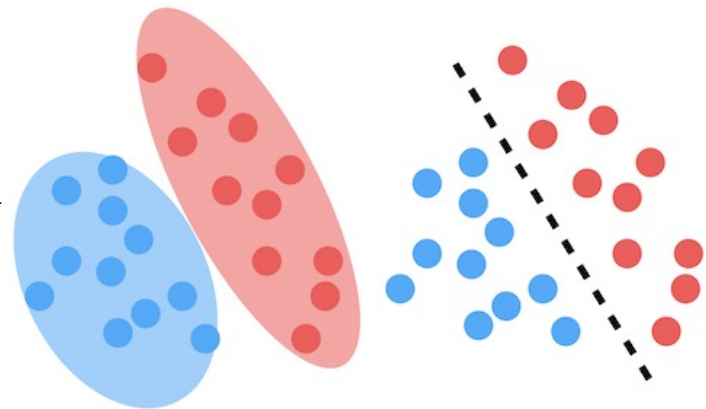
Generative vs. Discriminative Models

- **Generative Models (e.g., LDA):**
 - Model the data generative process
 - Estimate the prior $p_y(k)$ and the likelihood probability $p_{x|y}(\mathbf{x}|k)$ from the training data
 - Estimate $p_{y|x}(k|\mathbf{x})$ based on Bayes rule:

$$p_{y|x}(k|\mathbf{x}) = \frac{p_y(k) \cdot p_{x|y}(\mathbf{x}|k)}{p_x(\mathbf{x})}$$

- **Discriminative Models (e.g., Logistic Regression):**
 - Directly assume some functional form of $p_{y|x}(k|\mathbf{x})$
 - Estimate the parameters of $p_{y|x}(k|\mathbf{x})$ from the training data
 - Make decisions based on

$$f_{\star}(\mathbf{x}) = \begin{cases} 1 & \text{if } p_{y|x}(1|\mathbf{x}) \geq 1/2, \\ 0 & \text{if } p_{y|x}(1|\mathbf{x}) < 1/2 \end{cases}$$



Logistic Function: Gaussian Case

- For the binary case, suppose $\mathbf{x} \mid y = k \sim \mathcal{N}(\mathbf{u}_k, \Sigma)$
- By the Bayes rule, we have

$$\begin{aligned} p_{y|\mathbf{x}}(1 \mid \mathbf{x}) &= \frac{p_y(1)\psi(\mathbf{x}; \boldsymbol{\mu}_1, \Sigma)}{p_y(0)\psi(\mathbf{x}; \boldsymbol{\mu}_0, \Sigma) + p_y(1)\psi(\mathbf{x}; \boldsymbol{\mu}_1, \Sigma)}, \\ &= \frac{p_y(1)e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_1)^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_1)}}{p_y(0)e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_0)^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_0)} + p_y(1)e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_1)^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_1)}} \\ &= \frac{1}{1 + \frac{p_y(0)}{p_y(1)}e^{\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_1)^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_1) - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_0)^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_0)}}} \\ &= \frac{1}{1 + \exp(-(w^\top \mathbf{x} + b))} \end{aligned}$$

$w = \hat{\Sigma}^{-1}(\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0) \in \mathbb{R}^d$
 $b = \frac{1}{2}\hat{\boldsymbol{\mu}}_0^\top \hat{\Sigma}^{-1}\hat{\boldsymbol{\mu}}_0 - \frac{1}{2}\hat{\boldsymbol{\mu}}_1^\top \hat{\Sigma}^{-1}\hat{\boldsymbol{\mu}}_1 + \log \frac{\hat{p}_y(1)}{\hat{p}_y(0)}$

Logistic Regression

Take the binary classification problem as an example,

- Assume that

$$\eta(\mathbf{x}) := p_{y|x}(1 \mid \mathbf{x}) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x} + b))}, \quad \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}$$

- Directly estimate $\mathbf{w}$ and b from the data
- Plug the estimate

$$\hat{\eta}(\mathbf{x}) := \hat{p}_{y|x}(1 \mid \mathbf{x}) = \frac{1}{1 + \exp(-(\hat{\mathbf{w}}^\top \mathbf{x} + \hat{b}))}$$

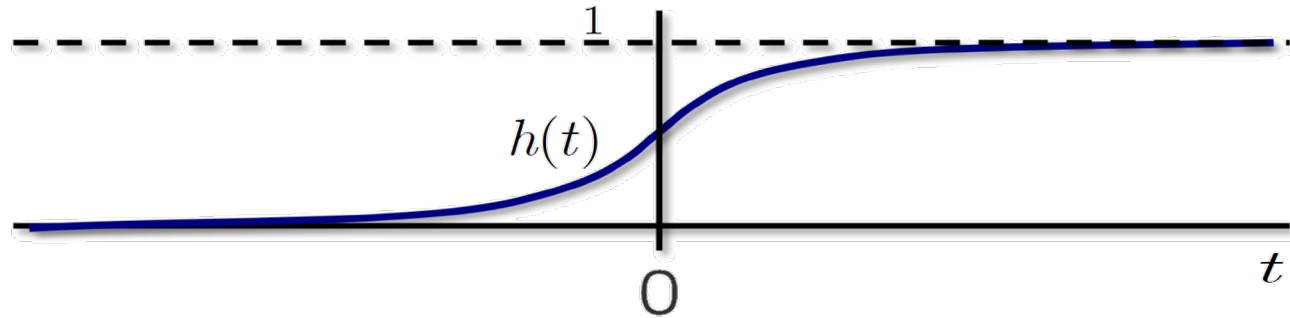
into the formula for the Bayes classifier.

Logistic Function

The function

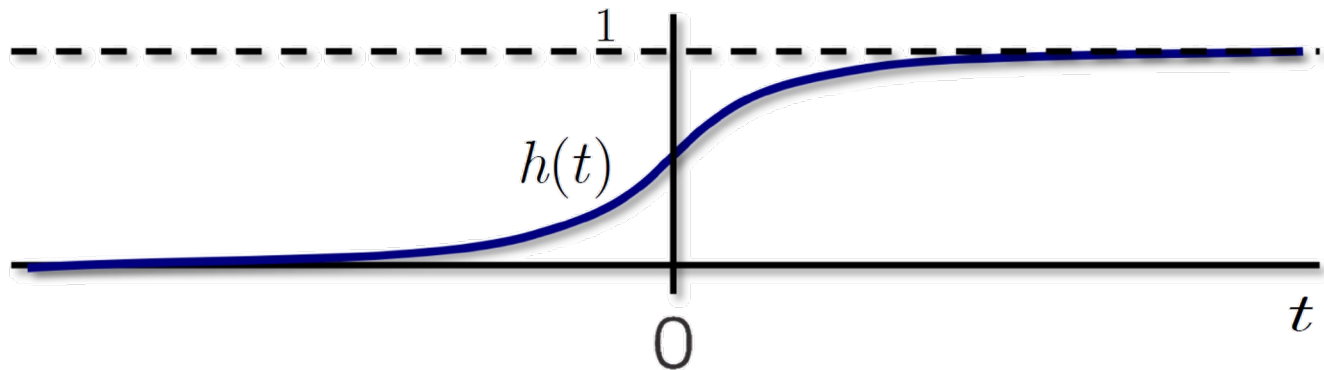
$$h(t) = \frac{1}{1 + e^{-t}}$$

is called the **logistic function** or **sigmoid**



Sigmoid: 'S'-like function

Logistic Function: Probability Interpretation



- $h(t) \in [0, 1]$ -- can be interpreted as probability.
- $\eta(\mathbf{x})$ is the probability that $y = 1$

$$\eta(\mathbf{x}) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x} + b))}, \quad \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}$$

Logistic Regression Classifier is a Linear Classifier

For simplicity, denote the logistic regression classifier by

$$\hat{f}(\mathbf{x}) = \mathbb{1}_{\hat{\eta}(\mathbf{x}) \geq 1/2}(\mathbf{x})$$

Notice that

$$\begin{aligned} \hat{f}(\mathbf{x}) = 1 & \iff \hat{\eta}(\mathbf{x}) \geq \frac{1}{2} \\ & \iff \frac{1}{1 + \exp\left(-(\hat{\mathbf{w}}^\top \mathbf{x} + \hat{b})\right)} \geq \frac{1}{2} \\ & \iff \exp\left(-(\hat{\mathbf{w}}^\top \mathbf{x} + \hat{b})\right) \leq 1 \\ & \iff \hat{\mathbf{w}}^\top \mathbf{x} + \hat{b} \geq 0 \end{aligned}$$

Logistic Regression Classifier is a Linear Classifier

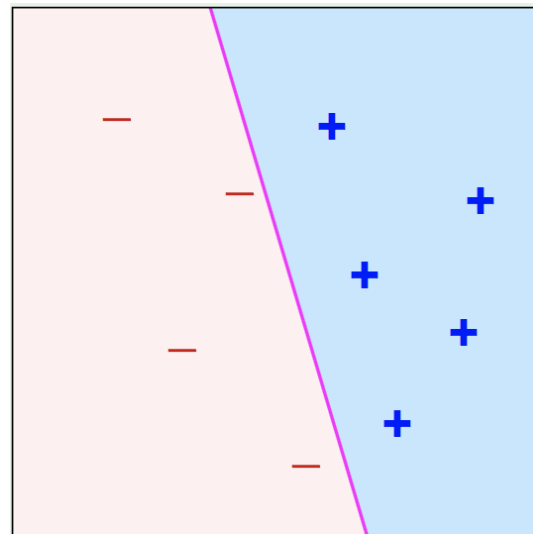
For simplicity, denote the logistic regression classifier by

$$\hat{f}(\mathbf{x}) = \mathbb{1}_{\hat{\eta}(\mathbf{x}) \geq 1/2}(\mathbf{x})$$

So that $\hat{f}(\mathbf{x}) = 1 \iff \hat{\mathbf{w}}^\top \mathbf{x} + \hat{b} \geq 0$

$$\hat{f}(\mathbf{x}) = \begin{cases} 1 & \text{if } \hat{\mathbf{w}}^\top \mathbf{x} + \hat{b} \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

linear classifier



Lecture Agenda

- Classification Problem & Bayes-Optimal Classifier
- Linear Discriminative Analysis & Linear Classifier
- **Logistic Regression Models**
 - Logistic Function & Linear Classifier
 - **Parameter Estimation for Logistic Regression**

Parameter Estimation

- **Given:** the training data

$$\mathcal{S} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$$

- **Challenge:** how to estimate the parameters $(\mathbf{w}, b)$ for

$$\eta(\mathbf{x}) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x} + b))} \quad ?$$

- **One possibility** (recall from LDA):

$$(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n) \xrightarrow{\text{blue arrow}} \hat{\boldsymbol{\mu}}_0, \hat{\boldsymbol{\mu}}_1, \hat{\boldsymbol{\Sigma}} \xrightarrow{\text{blue arrow}} \begin{aligned} \mathbf{w} &= \hat{\boldsymbol{\Sigma}}^{-1} (\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0) \in \mathbb{R}^d \\ b &= \frac{1}{2} \hat{\boldsymbol{\mu}}_0^\top \hat{\boldsymbol{\Sigma}}^{-1} \hat{\boldsymbol{\mu}}_0 - \frac{1}{2} \hat{\boldsymbol{\mu}}_1^\top \hat{\boldsymbol{\Sigma}}^{-1} \hat{\boldsymbol{\mu}}_1 + \log \frac{\hat{p}_y(1)}{\hat{p}_y(0)} \end{aligned}$$



Parameter Estimation

- **Given:** the training data

$$\mathcal{S} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$$

- **Challenge:** how to estimate the parameters $(\mathbf{w}, b)$ for

$$\eta(\mathbf{x}) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x} + b))} \quad ?$$

- **Alternative approach?**

- Why not directly estimate $(\mathbf{w}, b)$, since they have much smaller size than $\hat{\boldsymbol{\mu}}_0, \hat{\boldsymbol{\mu}}_1, \hat{\boldsymbol{\Sigma}}$?

Maximum likelihood estimation



Maximum Likelihood Estimation

- For convenience, let us denote by $\boldsymbol{\theta} = (\boldsymbol{w}, b)$
- Since

$$\eta(\boldsymbol{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^\top [\boldsymbol{x}, 1])} = \frac{1}{1 + \exp(-\boldsymbol{\theta}^\top \tilde{\boldsymbol{x}})}$$

We use the notation $\eta(\boldsymbol{x}; \boldsymbol{\theta})$ to highlight the dependence of $\eta(\cdot)$ on both $\boldsymbol{x}$ and $\boldsymbol{\theta}$, where we denote $\tilde{\boldsymbol{x}} := [\boldsymbol{x}^\top, 1]^\top$

- We assume that the data $\mathcal{S} = \{(\boldsymbol{x}_1, y_1), \dots, (\boldsymbol{x}_n, y_n)\}$ are *identical and independently distributed*.

The *a posteriori* Probability of Data

- Suppose we knew $\boldsymbol{\theta} = (\boldsymbol{w}, b)$, then we compute

$$\begin{aligned}\mathbb{P}(y_i \mid \boldsymbol{x}_i, \boldsymbol{\theta}) &= \begin{cases} \eta(\boldsymbol{x}; \boldsymbol{\theta}) & \text{if } y_i = 1 \\ 1 - \eta(\boldsymbol{x}; \boldsymbol{\theta}) & \text{if } y_i = 0 \end{cases} \\ &= \eta(\boldsymbol{x}_i; \boldsymbol{\theta})^{y_i} (1 - \eta(\boldsymbol{x}_i; \boldsymbol{\theta}))^{1-y_i}\end{aligned}$$

- Because of independence, we have

$$\begin{aligned}\mathbb{P}(y_1, \dots, y_n \mid \boldsymbol{x}_1, \dots, \boldsymbol{x}_n; \boldsymbol{\theta}) &= \prod_{i=1}^n \mathbb{P}(y_i \mid \boldsymbol{x}_i; \boldsymbol{\theta}) \\ &= \prod_{i=1}^n \eta(\boldsymbol{x}_i; \boldsymbol{\theta})^{y_i} (1 - \eta(\boldsymbol{x}_i; \boldsymbol{\theta}))^{1-y_i}\end{aligned}$$

The Log-likelihood

- The likelihood:

$$\ell(\boldsymbol{\theta}) = \prod_{i=1}^n \eta(\mathbf{x}_i; \boldsymbol{\theta})^{y_i} (1 - \eta(\mathbf{x}_i; \boldsymbol{\theta}))^{1-y_i}$$

- The log-likelihood:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n [y_i \log \eta(\mathbf{x}_i; \boldsymbol{\theta}) + (1 - y_i) \log (1 - \eta(\mathbf{x}_i; \boldsymbol{\theta}))]$$

- We have $\arg \max_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$

Simplification of the Log-likelihood

- The notation:

$$\boldsymbol{\theta} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}, \quad \tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$$

so that $\mathbf{w}^\top \mathbf{x} + b = \boldsymbol{\theta}^\top \tilde{\mathbf{x}}$

- Given $h(t) = \frac{1}{1 + e^{-t}}$, and $h(\boldsymbol{\theta}^\top \tilde{\mathbf{x}}) = \eta(\mathbf{x}_i; \boldsymbol{\theta}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i)}$
- Thus, we can rewrite the log-likelihood as

$$\begin{aligned} \mathcal{L}(\boldsymbol{\theta}) &= \sum_{i=1}^n [y_i \log h(\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i) + (1 - y_i) \log (1 - h(\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i))] \\ &= \sum_{i=1}^n [(y_i - 1) \boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i - \log (1 + \exp(-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i))] \end{aligned}$$

Maximizing the Log-likelihood

- How can we maximize

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n [(y_i - 1) \boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i - \log(1 + \exp(-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i))]$$

with respect to $\boldsymbol{\theta}$?

- **Possible idea:** find the $\boldsymbol{\theta}$ such that its gradient

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial}{\partial \theta_1} \mathcal{L}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_2} \mathcal{L}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_{d+1}} \mathcal{L}(\boldsymbol{\theta}) \end{bmatrix} = \mathbf{0}.$$

Computing the Gradient of the Logistic Loss

For our logistic loss

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n \left[(y_i - 1) \boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i - \log (1 + \exp (-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i)) \right]$$

Its gradient is given by

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) &= \sum_{i=1}^n \nabla_{\boldsymbol{\theta}} \left[(y_i - 1) \boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i - \log (1 + e^{-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i}) \right] \\ &= \sum_{i=1}^n \left[(y_i - 1) \tilde{\mathbf{x}}_i + \tilde{\mathbf{x}}_i e^{-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i} (1 + e^{-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i})^{-1} \right] = \sum_{i=1}^n \tilde{\mathbf{x}}_i (y_i - h(\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i)) \end{aligned}$$

Thus, $\sum_{i=1}^n \tilde{\mathbf{x}}_i (y_i - h(\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i)) = \mathbf{0}$ gives $d+1$ **nonlinear** equations, but it has **no closed-form** solutions (compared to linear regression)

Maximizing the Log-likelihood

- How can we maximize

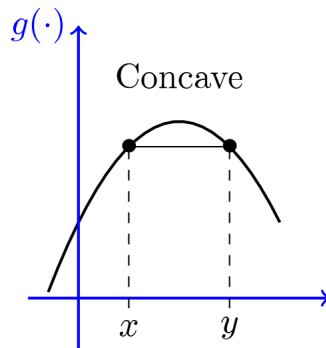
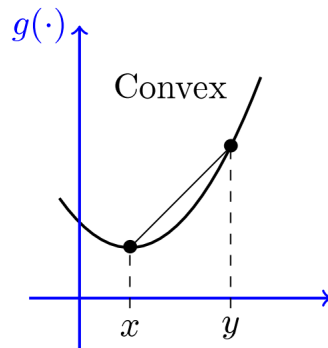
$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n [(y_i - 1) \boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i - \log (1 + \exp (-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i))]$$

with respect to $\boldsymbol{\theta}$?

- The learning problem:

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} -\mathcal{L}(\boldsymbol{\theta})$$

- Bad news: **no** closed-form solution
- Good news: the function is convex, we can obtain it via **optimization**

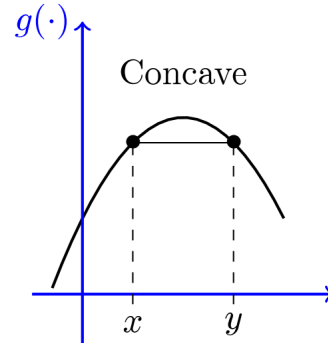
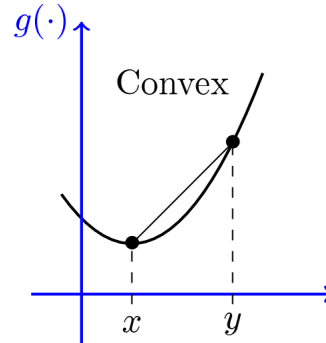


Optimizing the MLE Loss

Throughout machine learning (and many other areas), we will often encounter problems of the form

$$\min_{\mathbf{z} \in \mathbb{R}^d} g(\mathbf{z}), \quad \text{s.t. } \mathbf{z} \in \mathcal{C}.$$

(e.g., $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} -\mathcal{L}(\boldsymbol{\theta})$ for today)



- Our function is **convex** (e.g., every local minimizer is global)
- Both **gradient descent** and **Newton's methods** are common strategies for optimizing the parameters

Example: Gradient Descent

To solve the optimization problem

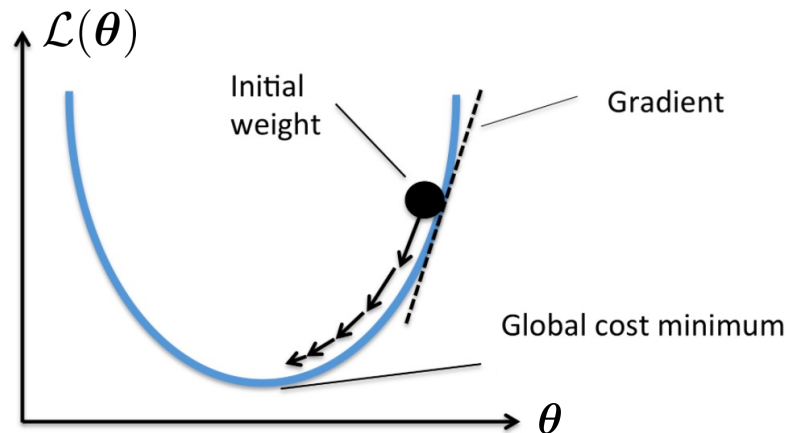
$$\min_{\boldsymbol{\theta}} -\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^n [(y_i - 1) \boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i - \log(1 + \exp(-\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i))]$$

A vanilla approach is taking gradient descent

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \tau_k \cdot \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$$

Notations:

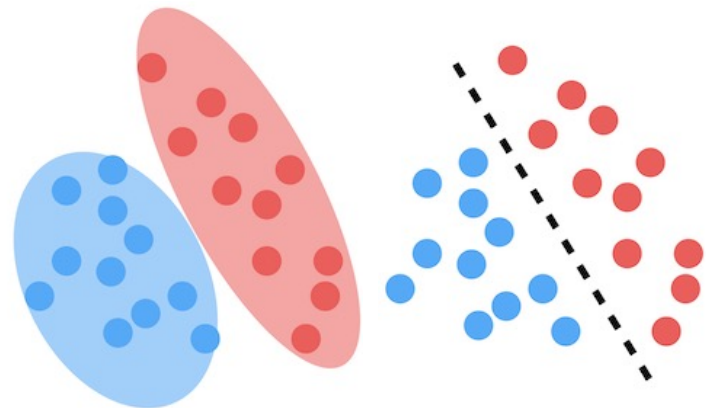
- Current and next iterates: $\boldsymbol{\theta}_k, \boldsymbol{\theta}_{k+1}$
- Stepsize τ_k , gradient $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$



Summary: Generative vs. Discriminative Methods

Both **LDA** and **logistic regression** classifiers are **linear classifiers**

- **Linear discriminant analysis:**
 - Based on Gaussian generative model
 - Needs to estimate the joint distribution of $(\mathbf{x}, y)$ from the data
- **Logistic regression:**
 - Model only the distribution of $y \mid \mathbf{x}$, not $(\mathbf{x}, y)$
 - Valid for a larger class of distributions
 - Fewer parameters to estimate



Further Readings & References

- [The Elements of Statistical Learning: Data Mining, Inference, and Prediction](#), by Trevor Hastie, Robert Tibshirani, and Jerome Friedman (Chapter 4)